MACHINE LEARNING PROJECT REPORT

Review1:ExploratoryDataAnalysis&DataPreprocessing

SOFTWARE DEFECT PREDICTION USING SOFTWARE METRICS

Dataset: NASA–PROMISE KC1 Software Defect Dataset

Course: Machine Learning

Submitted to: Mr. Nirbhay

Team members: Bhumi Jain- 22615990002

Submission date: 26 October2026

Abstract

Thisreportpresents the exploratory data analysis (EDA) and preprocessing of a software defect prediction dataset identified as NASA–PROMISE KC1. Each record represents a software module described by software metrics, while the target column defects indicates whether a module is labelled defective or non-defective. The analysis examines missing-value patterns, record cleaning, target distribution, potential outliers, feature correlations, and the relationship between cyclomatic complexity and defect status. The report documents Review 1 only; it does not claim predictive model performance.

1. Introduction

Software defect prediction uses software-module characteristics to estimate whether a module may contain defects. This can help development teams prioritize testing, code review, and quality assurance. Before training a classifier, the data should be inspected, cleaned, and transformed consistently.

1.1 Problem Statement

Prepare and explore software metrics for a binary classification task in which defects is the target label: non-defective or defective.

1.2 Objectives

Inspect dataset structure, data quality, distributions, and target classes.
Visualize missing-value patterns and record counts before and after cleaning.
Identify duplicate records and potential outliers in numerical metrics.
Examine relationships among software metrics and defect labels.
Document target encoding, outlier treatment, and feature scaling.
Provide reproducible code and a clear report in a public GitHub repository.

2. Dataset Overview

The dataset is identified as the NASA–PROMISE KC1 software defect dataset. Rows represent software modules and columns represent software metrics. The target column is defects, represented in the supplied target plot as 0 = No Defect and 1 = Defect. The metrics shown include loc (lines of code), v(g) (cyclomatic complexity), ev(g), iv(g), Halstead-related measures such as n, v, l, d, i, e, b and t, lines-of-code measures, operator/operand counts, and branchCount.

Use exact column names from the CSV in the final repository. Some columns appear blank in the correlation heatmap; check whether they are constant, all-missing, or otherwise unsuitable for correlation calculations.

3. Tools and Workflow

The suppliedvisualizations indicate a Python-based analysis. Include the exact libraries and versions used in the submitted code. A typical workflow is: load dataset → inspect and clean invalid values → examine missingness and duplicates → encode target → detect/treat outliers → scale numerical features → save output and figures.

4. Exploratory Data Analysis (EDA)

EDA wasused tounderstand the dataset's quality,class balance, feature distributions, and relationships before model development. The following figures are inserted from the supplied images. Interpretations are based on visible patterns; confirm exact values using the code outputs.

4.1 Missing-value analysis

Theheatmap appears uniformly colored, so missingness cannot be reliably inferred from color alone. Confirm missing values using per-column numerical counts.

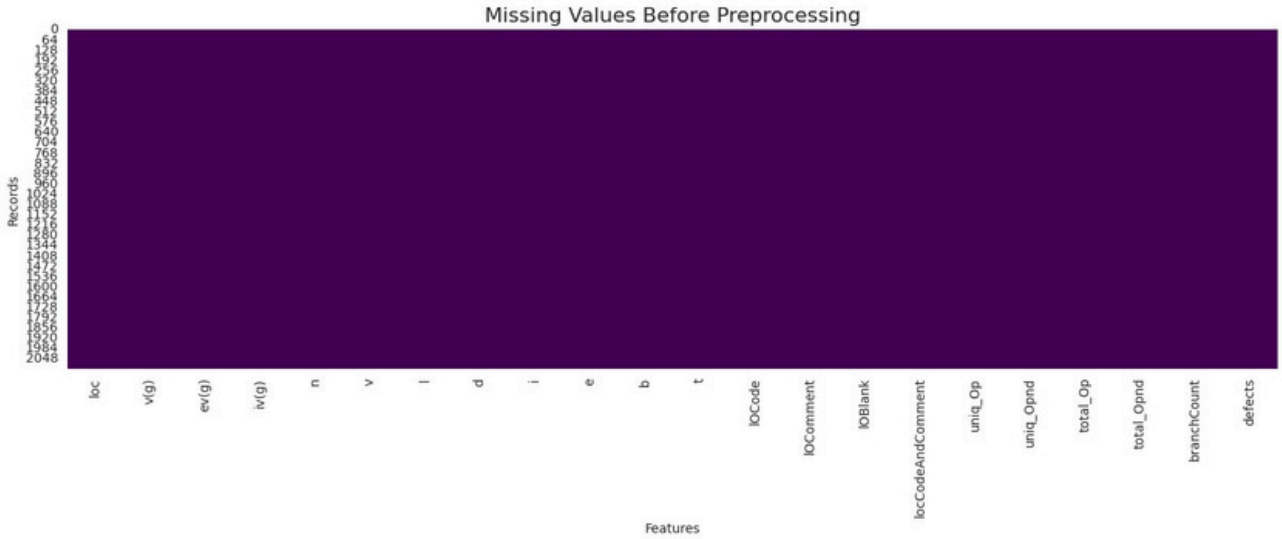


Figure 1. Missing-value overview before preprocessing.

Both panels appear uniform. Report the actual before/after missing-value countsfrom the code; do not claim that imputation changed the data unless the counts confirm it.

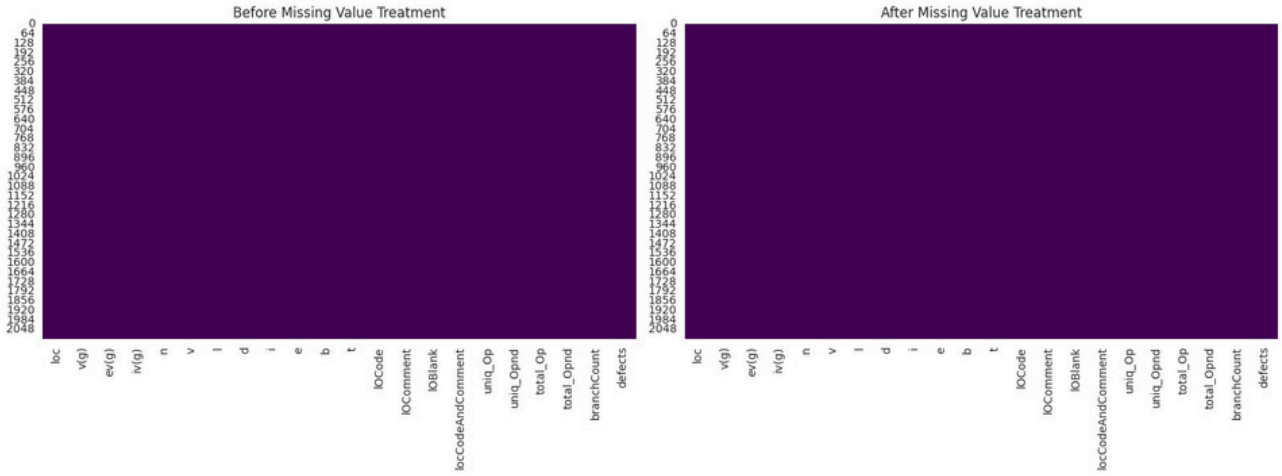


Figure 2. Missing-value heatmaps before and after treatment.

4.2 Record cleaning and duplicates

Thechartshows2,109recordsbeforecleaning and 1,212 afterward—a reduction of 897 records (about 42.5%). Verify whether these were all exact duplicates or whether other cleaning filters were applied.

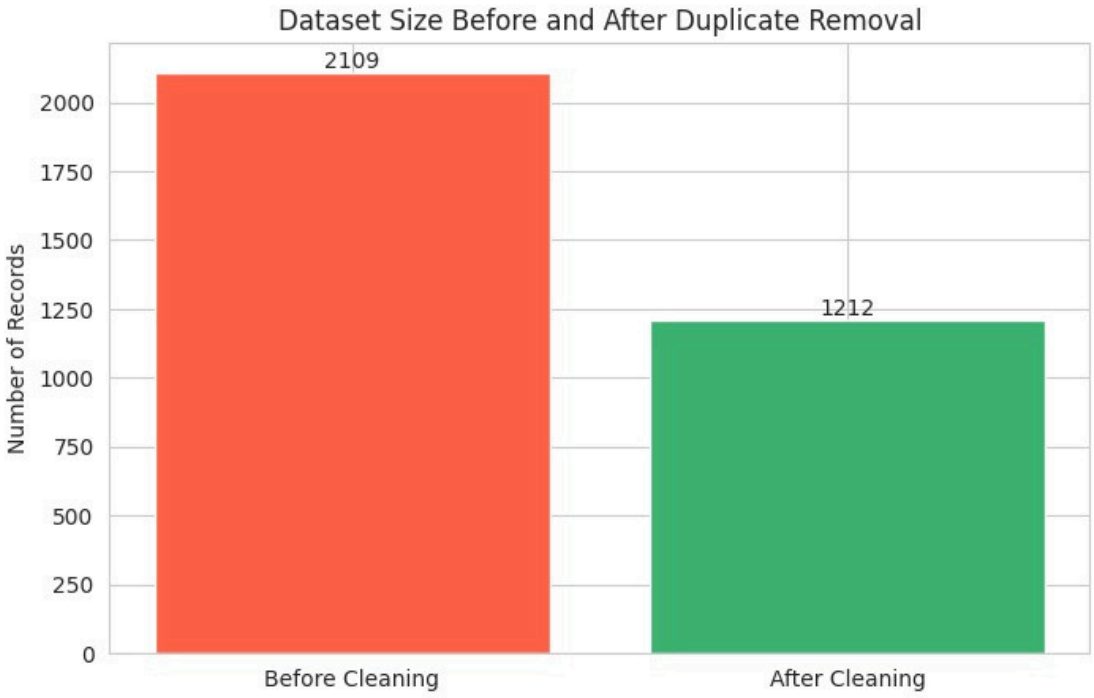


Figure 3. Record count before and after cleaning.

4.3 Target-class distribution

The chart shows approximately900 non-defective modules and 310 defective modules. The target is imbalanced, with non-defective modules forming the majority class. Class imbalance matters during model evaluation: accuracy alone can be misleading. When a model is trained, also review precision, recall, F1-score, and the confusion matrix.

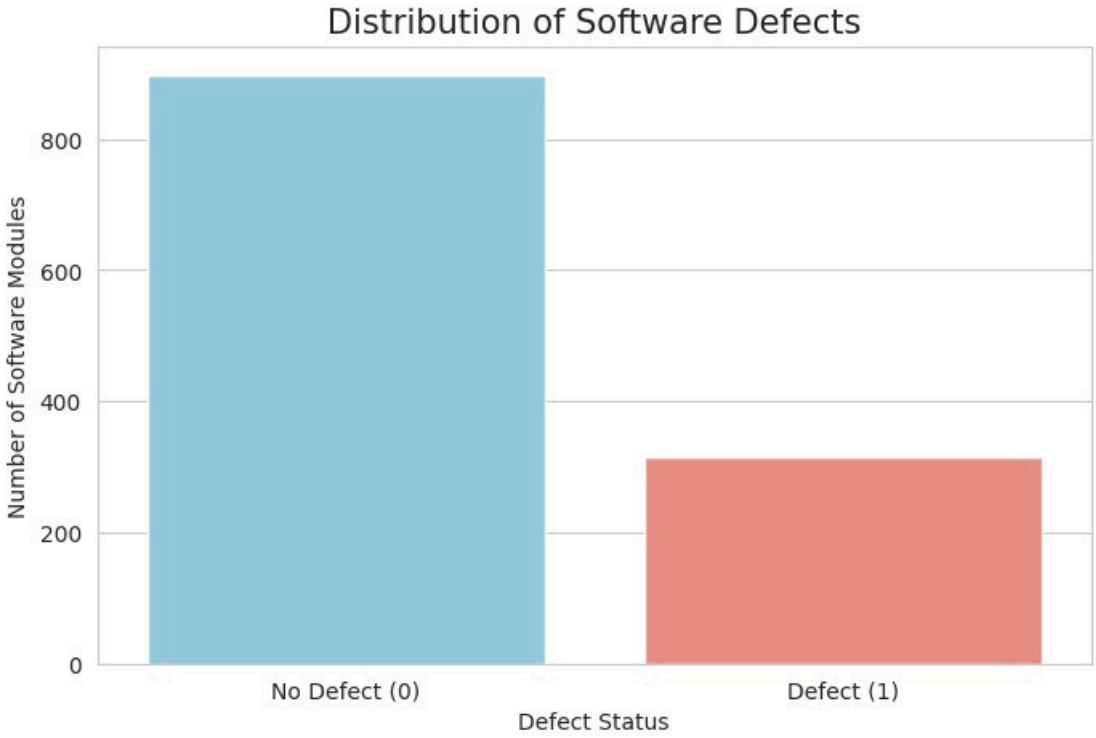


Figure 4. Distribution of software defect labels.

4.4 Outlier detection

Aboxplot's central linerepresents the median, the box represents the interquartile range (Q1–Q3), whiskers show the range under the plotting rule, and points beyond whiskers indicate potential outliers.

The boxplots show many high-value points for loc, v(g), v, and e. These are potential outliers, but may represent legitimately large or complex modules rather than data errors.

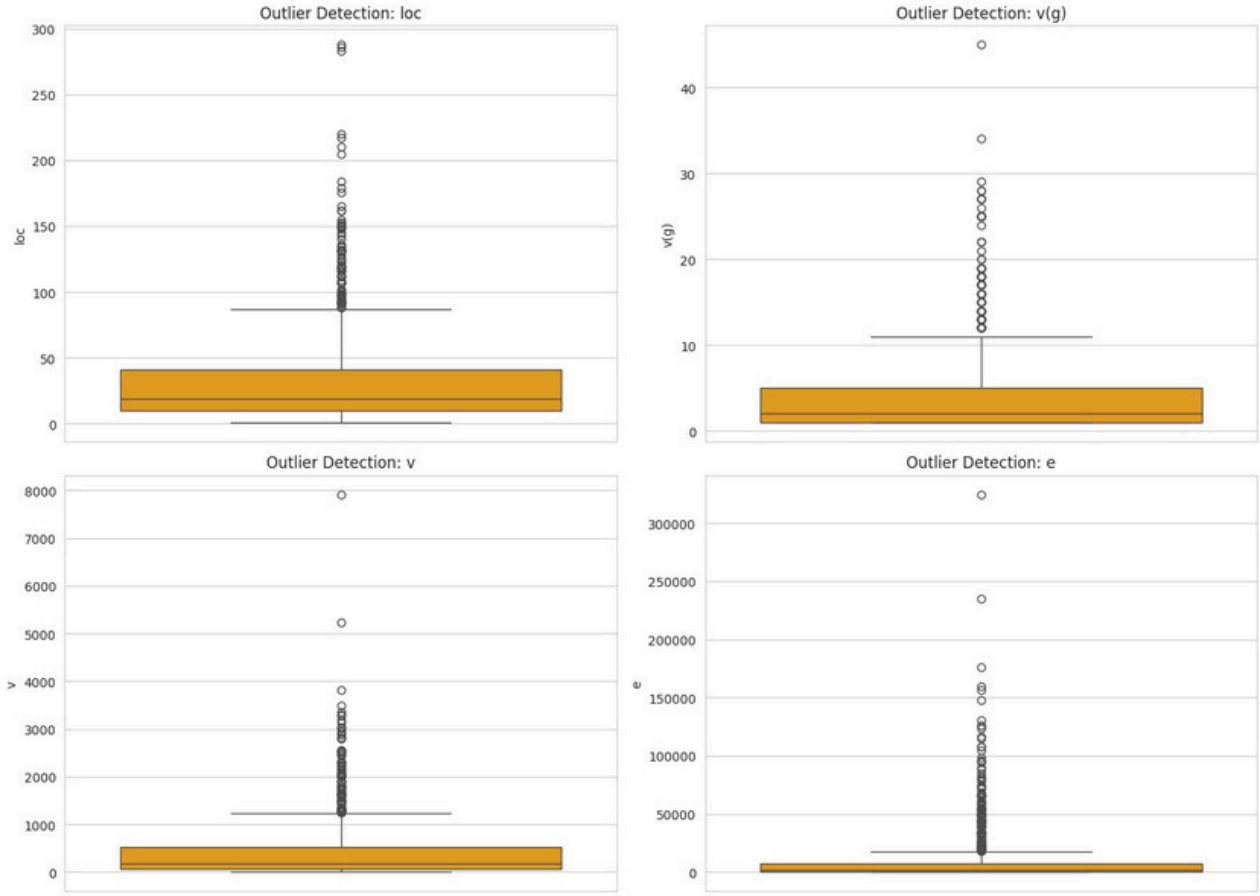


Figure 5. Potential outliers in selected software metrics before treatment.

The upper extremes are constrained after treatment, consistent with IQR-based capping.Capping changes values but does not remove the corresponding records; match this description to the code.

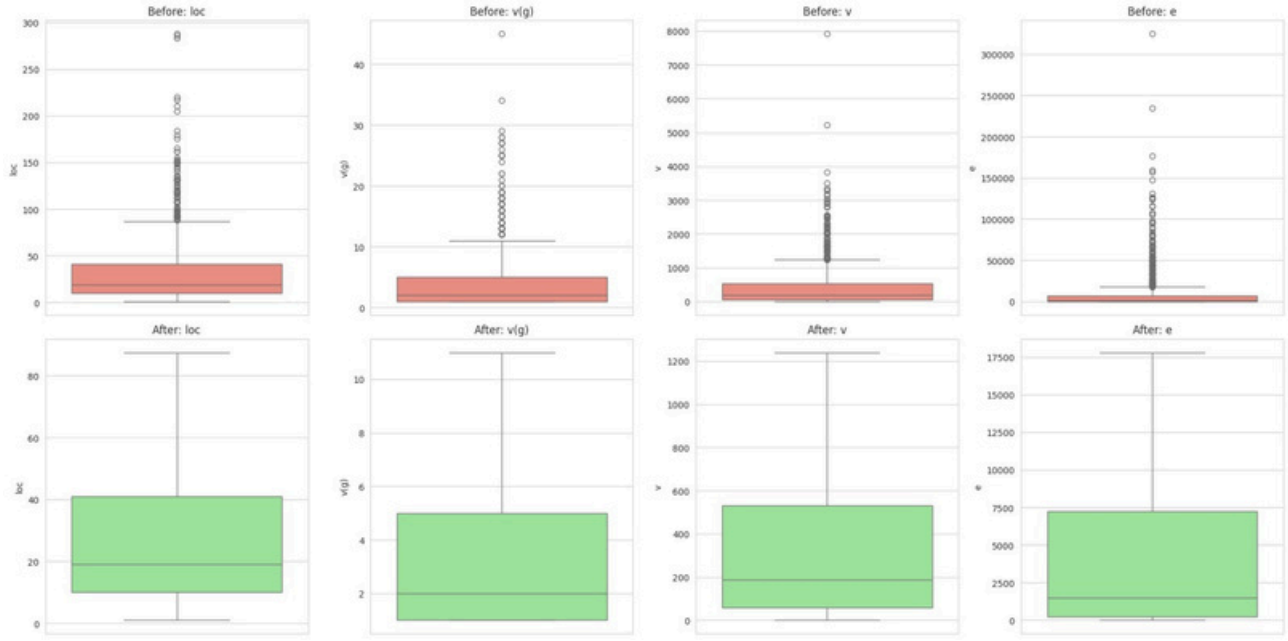


Figure 6. Selected metric distributions before and after outlier treatment.

4.5 Feature scaling

Beforescaling, the selected metrics have very different ranges. After scaling, their values occupy a more comparable range. If StandardScaler was used, each feature is transformed as z = (x − mean) / standard deviation. This centers the mean near zero and scales standard deviation near one on the data used to fit it; it does not necessarily make the median zero.

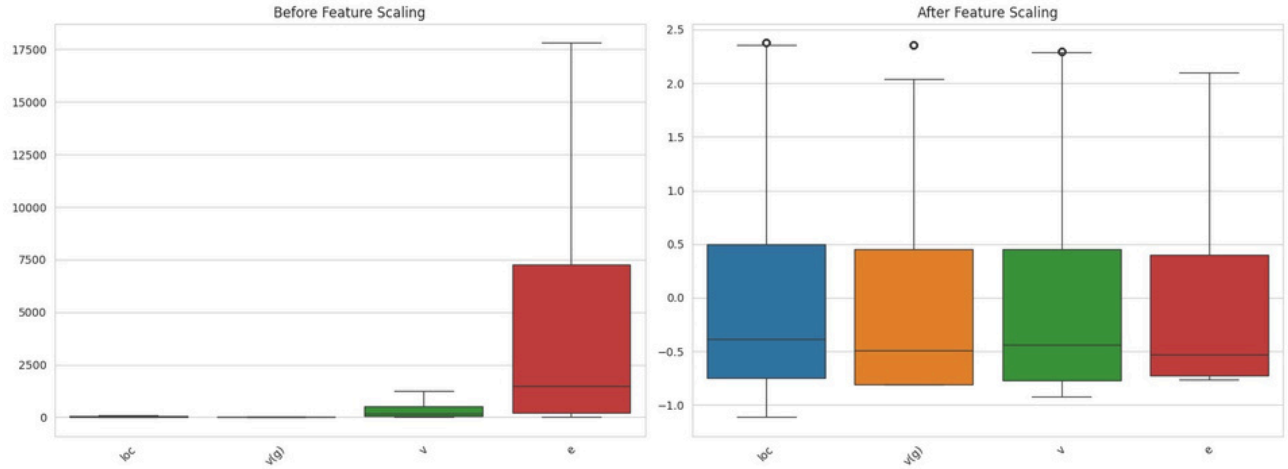


Figure 7. Selected features before and after scaling.

4.6 Correlation analysis

Severalmetrics are strongly positively correlated; n and v appear near 1.00. The l metric is negatively correlated with many others. Blank cells may indicate constant or missing-valued features; verify them. Correlation does not prove causation, and weak linear correlation does not necessarily mean a feature has no predictive value.

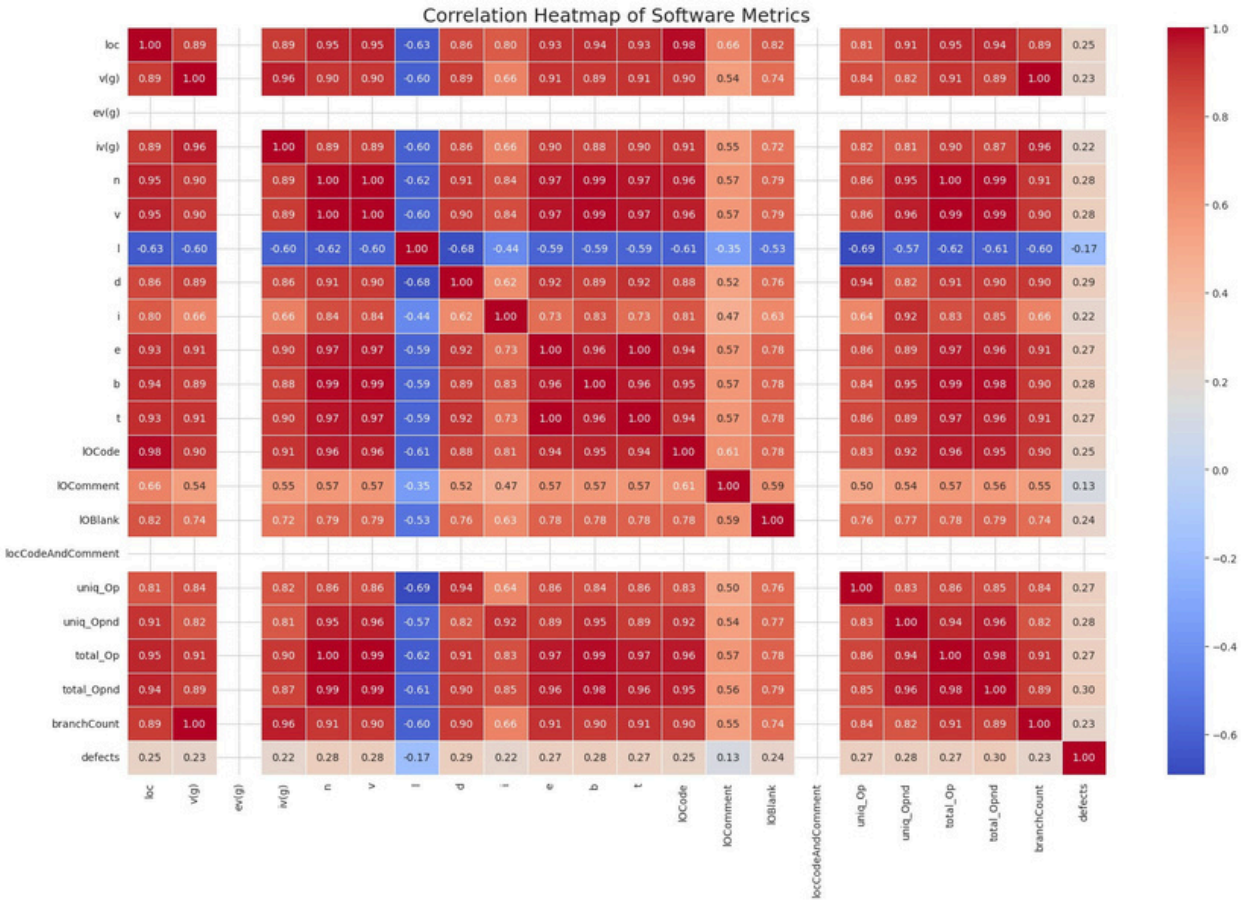


Figure 8. Correlation heatmap of software metrics.

4.7 Cyclomatic complexity and defect status

Thedefectivegrouphas ahigher mediancyclomatic complexity (about 4) than the non-defective group (about 2), though the distributions overlap. This suggests association, not proof of causation.

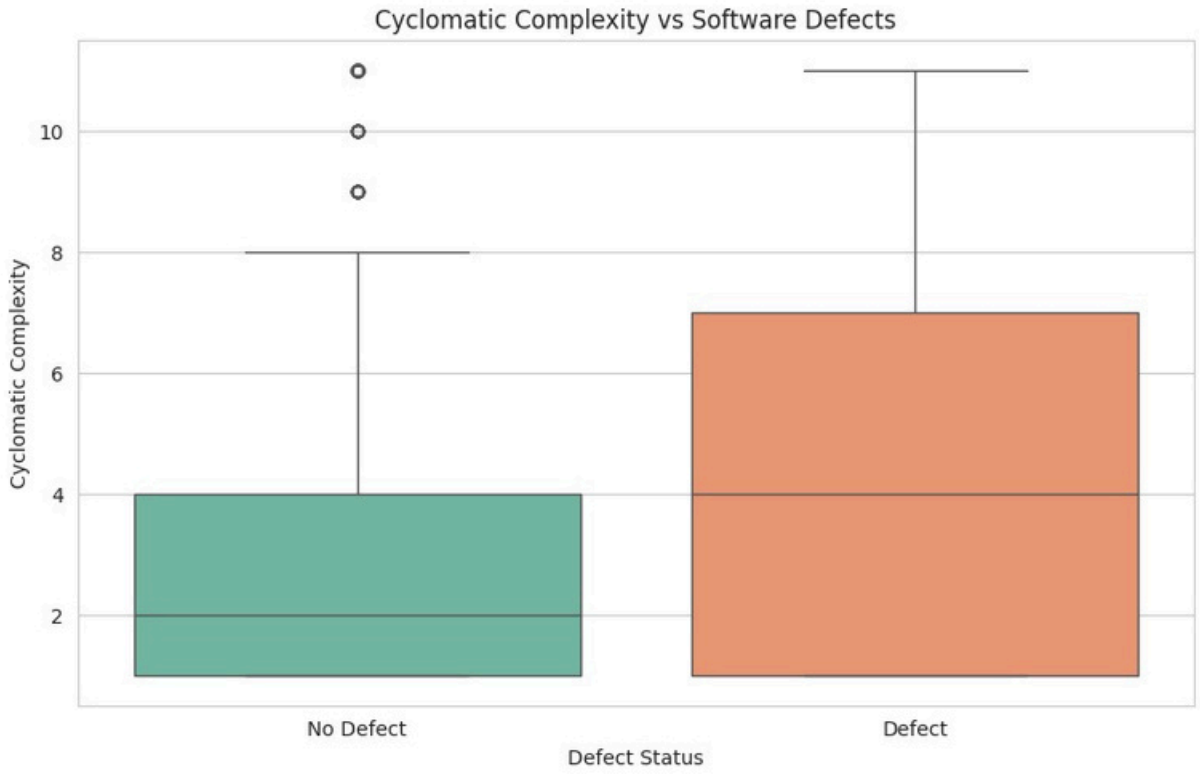


Figure 9. Cyclomatic complexity by defect status.

5. Data Preprocessing

The exact operationsandcountsin this section must match the submitted code. Use the subsections below as the report narrative and revise any method that differs from the actual implementation.

5.1 Data validation and numeric conversion

Inspect column names, data types, invalid placeholders, and missing values. Convert metric columns to numeric types before computing statistics, correlations, outlier thresholds, or scaling. Only treat markers such as '?', 'NA', or empty strings as missing if they occur in the source dataset.

5.2 Missing-value treatment

Count missing values for every column before choosing a treatment. Median imputation fills a missing numeric value with that feature's median and is less sensitive to extreme values than mean imputation. The supplied heatmaps are visually uniform, so use actual per-column counts to establish whether missing values existed and whether treatment changed them.

5.3 Duplicate handling

Exact duplicate rows can be identified and removed. The chart reports a reduction from 2,109 to 1,212 records, but the 897-record difference should be described as records removed during cleaning unless the code confirms that every removed row was an exact duplicate. Include the actual duplicate count from the code.

5.4 Target encoding

The target is represented as 0 for non-defective and 1 for defective. This is binary target encoding. Keep defects separate from the input feature matrix and do not scale the target as a numerical feature.

5.5 Outlier detection and treatment

The Interquartile Range(IQR) rule uses IQR = Q3 − Q1, with lower and upper fences Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. Values beyond the fences are flagged as potential outliers. The before/after plots suggest capping of extreme values. If the code clips values to the IQR fences, call this IQR-based outlier capping (winsorization), not row removal. If rows are dropped instead, revise this section and report the number removed.

5.6 Scaling and standardization

Standardizationtypically transforms each feature using z = (x − μ) / σ, where μ is the feature mean and σ is its standard deviation in the fitting data. This makes different numerical scales more comparable and can help scale-sensitive algorithms. Confirm the exact scaler in the code.

5.7 Avoiding data leakage

Forlater model evaluation, split data into training and test sets before fitting imputation, outlier thresholds, or scaling. Fit transformations on training data only and apply those fitted transformations to the test data. Fitting transformations on the full dataset before splitting can leak information from the test set. If the current script only creates a cleaned dataset, document this as a next-stage improvement.

5.8 Summary of preprocessing steps

Operation	*Method / evidence*	*Purpose*
Validation	Check types and invalid markers	Ensure data is interpreted correctly.
Missing values	Count per column; impute if required	Handle incomplete observations.
Record cleaning	Check exact duplicates	Avoid repeated records.
Target encoding	0 = No Defect; 1 = Defect	Represent binary labels numerically.
Outliers	IQR detection; plots suggest capping	Limit influence of extreme values.
Scaling	Confirm scaler used in code	Make numerical scales comparable.

6. Key Findings

The record-cleaning chart shows 2,109 rows before and 1,212 afterward; verify why 897 rows were removed.
The target is imbalanced, with non-defective modules forming the majority class.
Selected metrics show many high-value potential outliers before treatment.
Post-treatment boxplots show constrained upper values, consistent with capping.
Several software metrics are strongly correlated, suggesting overlapping information.
The defective group has a higher median cyclomatic complexity in the supplied boxplot, but the groups overlap.
The missing-value heatmaps do not visibly establish whether missing values were present; numerical counts are needed.

7. Limitations and Further Work

Verify all counts and methods against code output before submission.
Check blank correlation rows/columns for constant or missing-valued features.
Do not assume every outlier is an error; high metric values may be legitimate.
Use a stratified train/test split where appropriate because target classes are imbalanced.
During model evaluation, report precision, recall, F1-score, confusion matrix, and suitable additional metrics alongside accuracy.
Review highly correlated or constant predictors using a justified feature-selection process.

8. Conclusion

This reportdocuments EDA and preprocessing for the KC1 software defect prediction dataset. The supplied graphs indicate target-class imbalance, potential outliers in selected software metrics, strong correlations among some metrics, and a higher median cyclomatic complexity for defective modules. The substantial reduction in record count must be reconciled with the exact cleaning code. Missing-value status and preprocessing methods should be supported by numerical output. These steps provide a foundation for subsequent supervised learning, provided that the pipeline is reproducible and avoids data leakage.